Software Architecture and Design Specification

Project: Railway Reservation System (RRS) — Team 5, UE24CS341A

Version: 1.0

Authors: Ridhima Jain, S. S. Adhithya Sriram, Saathvik Shenoy, Padubidri, and Samhitha S.

Date: 28-09-2026

Revision History

Version	Date	Author	Change Summary
1.0	28-09-2026	Team 5	Initial architecture and design specification

Approvals

Role	Name	Signature/Date
Course Coordinator		
Team Lead	Samhitha S	

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Railway Reservation System (RRS), a C/C++ backend application with an ncurses Text User Interface (TUI) that automates train scheduling, ticket booking, waitlist/queue management and administrative operations. It translates the approved SRS v1.1 requirements into an implementable component structure, module decomposition, API contracts and UML models.

1.2 Scope

Covers the four owned functional modules (User Access & Profile, Inventory & Schedule, Search/Fare/Eco-Optimization, Booking & Queue Management), their internal architecture, the facade API that fronts them, persistence, the mock payment interface and the TUI presentation layer. Excludes real payment gateway integration, GPS tracking and native mobile apps, which are out of scope per SRS §2.3.

1.3 Audience

The members of Team 5 (developers/testers), the course instructor and teaching assistants (evaluators), and any future maintainer of the codebase , the course instructor and teaching assistants (evaluators), and any future maintainer of the codebase.

1.4 Definitions

PNR, RBAC, TUI, ncurses, B-tree, waitlist, eco-score, multi-leg journey — see SRS Appendix A for full definitions.

2. Document Overview

2.1 How to use this document

Section 3 (Architecture) describes the system's logical structure, module ownership, technology choices and security architecture. Section 4 (Design) provides UML sequence diagrams, API definitions and error handling needed to implement each module. Developers should read §3–§4 closely before implementing their owned module; evaluators should read §3.8 and §4.6 to trace design back to requirements.

2.2 Related Documents

RRS Synopsis v1.0, RRS SRS v1.1, RRS Validation Specification v1.0 (Test Plan).

3. Architecture

3.1 Goals & Constraints

Goals: correctness of booking/waitlist state, testability of the core engine independent of the TUI (NFR-007), maintainability through independently owned modules (NFR-006), and search performance under NFR-001 (95% of searches ≤ 2s for 200 stations / 500 trains).

Constraints: core business logic must be implemented in C/C++ (SRS §2.6); route search must use graph-based structures and route/schedule storage must use tree-based structures (B-trees); no real payment gateway is permitted; the system must be demonstrable via a working TUI in a terminal environment.

3.2 Stakeholders & Concerns

Stakeholder	Concern
Passengers	Fast, correct search and booking; clear waitlist status
Railway Administrators	Reliable inventory management; accurate operational reports
Course Instructor / TAs	Traceability from requirement to design to code to test; individual contribution evidence

Team 5 developers	Modularity — four owners must work in parallel without merge conflicts

3.3 Component (UML) Diagram

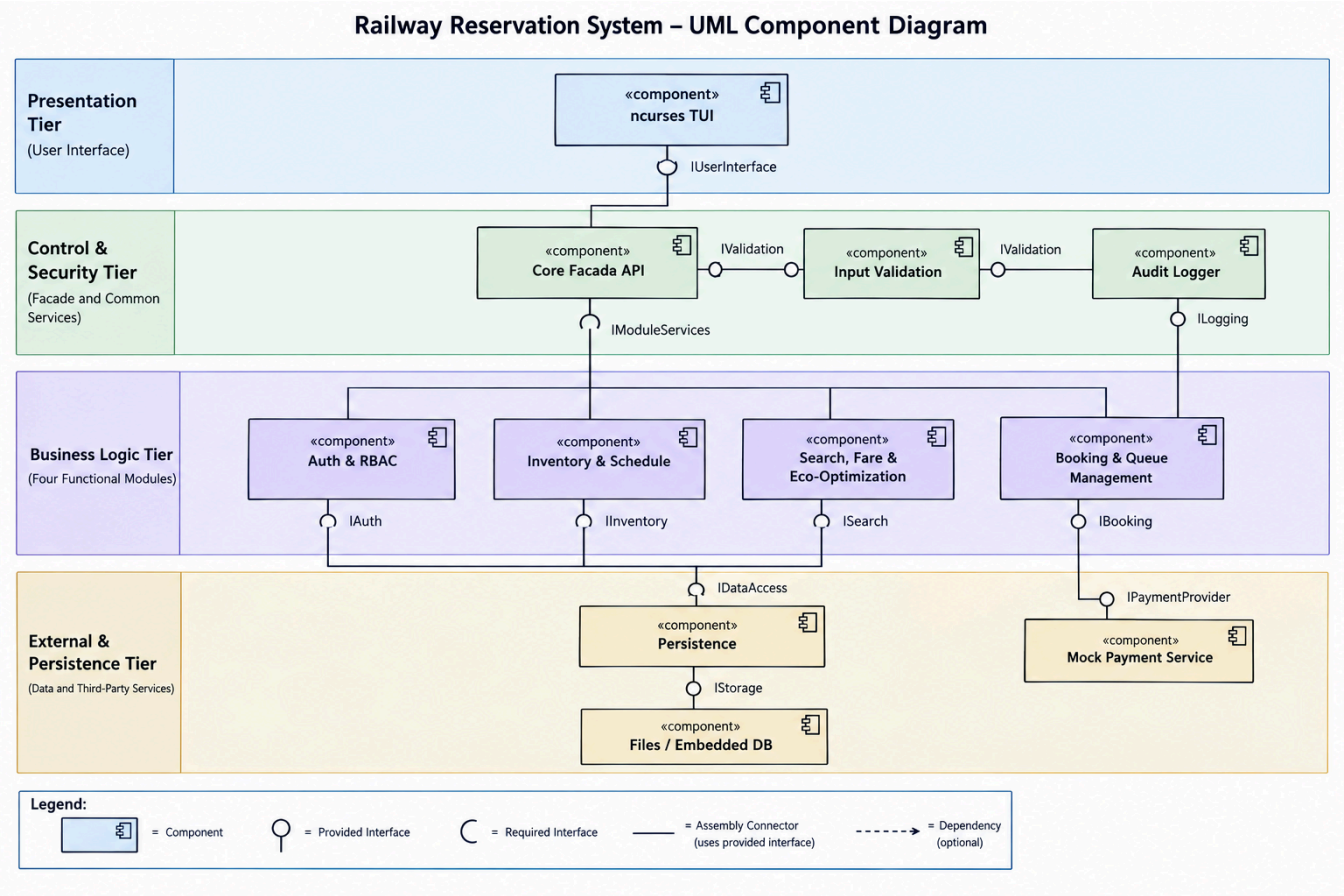


Figure 1: RRS component diagram — layered architecture with a facade API

3.4 Component Descriptions

- ncurses TUI: menu-driven interface for Passenger and Administrator roles; optionally fronted by a thin REST API wrapper for a MERN client.
- Core Facade API: single entry point that validates session and role before dispatching to a module (SEC-002, SEC-004).
- Input Validation: sanitizes and validates all user-supplied fields before they reach business logic (SEC-003).
- Auth & RBAC (Ridhima): registration, salted-hash login, session issuance, role-based access control.
- Inventory & Schedule (Adhithya): train, route, halt and per-class capacity management.
- Search, Fare & Eco-Optimization (Samhitha): graph-based route search, dynamic fare calculation, eco-score ranking.
- Booking & Queue Management (Saathvik): PNR generation, seat decrement, waitlist enqueue/promote.
- Mock Payment Service: simulates authorization and returns success/failure only; stores no card data (SEC-005).
- Audit Logger: records administrator actions and failed logins (SEC-006).
- Persistence: file storage or a lightweight embedded database behind a common interface.

3.5 Chosen Architecture Pattern and Rationale

A layered architecture with a facade API was chosen for RRS. It gives a single choke point for authentication and authorization (supporting SEC-002/SEC-004), keeps each of the four modules independently testable without the ncurses TUI (NFR-007), and lets four owners develop and commit against stable module boundaries in parallel (NFR-006). A microservices split was considered and rejected: at this scale it would add process/network overhead and CI complexity with no benefit, and the course's C/C++ constraint favors in-process module boundaries over network services.

3.6 Technology Stack & Data Stores

Area	Choice
Core engine	C / C++
UI	ncurses TUI (optional decoupled MERN/API wrapper)
Persistence	Flat files or a lightweight embedded database
Version control / Agile	Git, GitHub, GitHub Projects
Build & CI/CD	Make/CMake, GitHub Actions, Docker
Testing & static analysis	gtest/Unity or Check, gcov/lcov, cppcheck, Valgrind

3.7 Risks & Mitigations

Risk	Mitigation
ncurses unavailable on the evaluator's machine	Fall back to a plain CLI build path
Sprint 3 schedule slip (booking/waitlist is the most complex module)	Start graph and queue data structures in Sprint 2
Merge conflicts across four owners	Per-module source directories, small PRs, one reviewer from another module
Uneven Git commit distribution across members	Track commits per member weekly against the WBS

3.8 Traceability to Requirements

Component	Requirements covered
Auth & RBAC	FR-001 – FR-006, SEC-001, SEC-002, SEC-004
Inventory & Schedule	FR-007 – FR-011
Search, Fare & Eco	FR-012 – FR-017, NFR-001
Booking & Waitlist	FR-018 – FR-024, NFR-002, NFR-005, NFR-009
Persistence / Audit Logger	SEC-006, NFR-008
Facade / Input Validation	NFR-006, NFR-007, SEC-003

3.9 Security Architecture

Security objectives: SO-1 protect credential/data confidentiality; SO-2 restrict privileged actions to authorised roles; SO-3 preserve booking integrity; SO-4 provide auditability.

Threat (STRIDE)	Mitigation
Spoofing	Salted-hash password verification; session tokens bound to role
Tampering	Server-side validation on every write; capacity checks reject invalid state (FR-011)

Repudiation	Audit log of admin actions and failed logins (SEC-006)
Information Disclosure	No plaintext passwords (SEC-001); mock payment stores only a success/fail flag (SEC-005)
Denial of Service	Bounded search response time (NFR-001); input validation rejects malformed/oversized fields
Elevation of Privilege	Role-Based Access Control enforced at the facade on every protected call (SEC-002)

4. Design

4.1 Design Overview

RRS is designed as four loosely-coupled C/C++ modules behind a common facade, each exposing a small function-call API (see §4.3) so it can be unit-tested without the ncurses TUI or a live payment service. State transitions for bookings (WAITLISTED / CONFIRMED / CANCELLED) are centralized in the Booking module to keep waitlist promotion consistent and race-free.

4.2 UML Sequence Diagrams

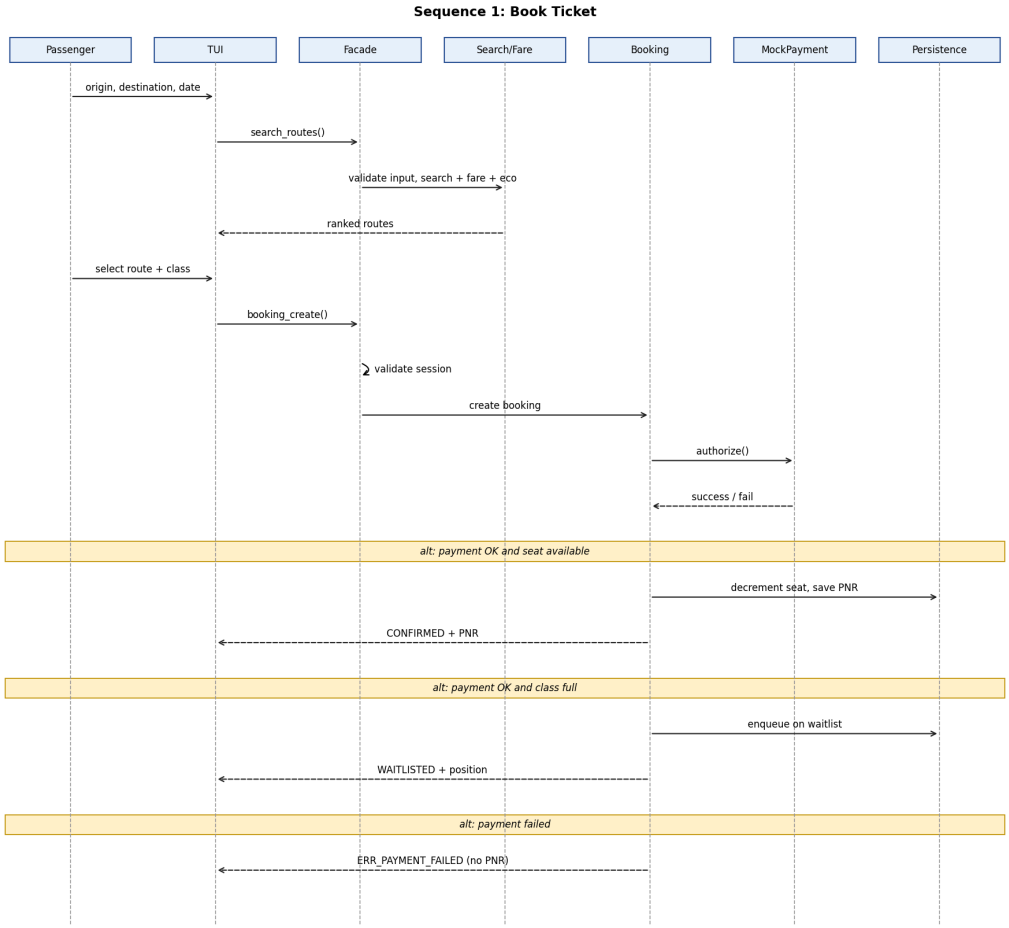


Figure 2: Sequence — Book Ticket (search, class selection, mock payment, confirm or waitlist)

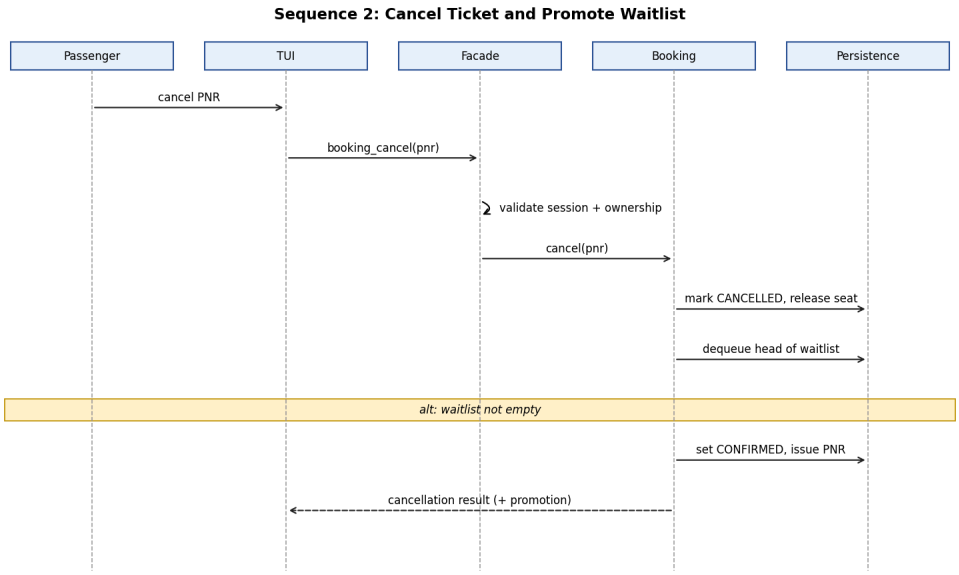


Figure 3: Sequence — Cancel Ticket and automatic waitlist promotion

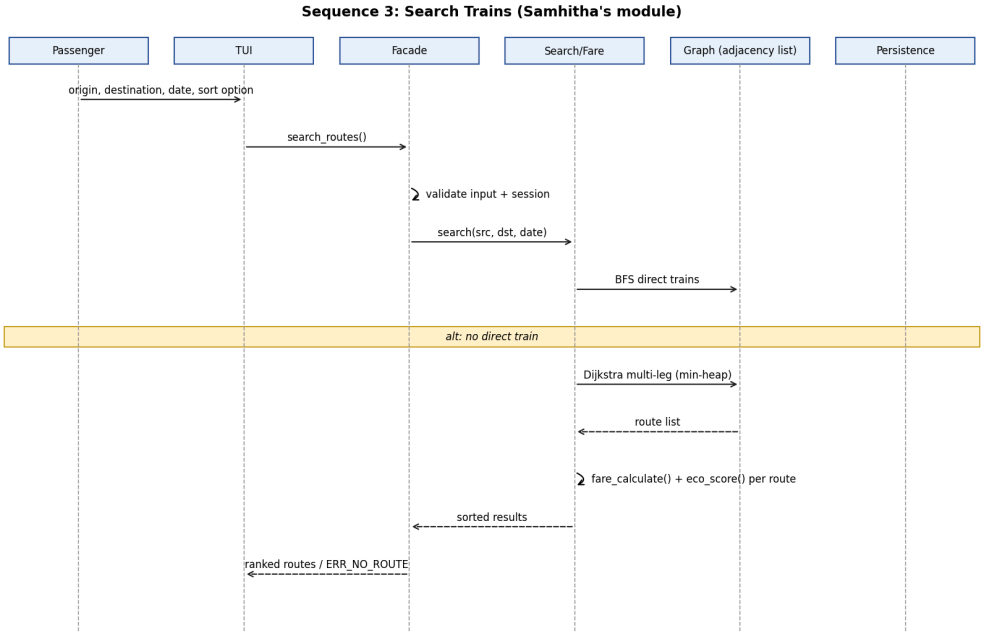


Figure 4: Sequence — Search Trains (Samhitha's module: graph search, fare and eco-score)

4.3 API Design

Interface definitions for the four owned modules (C function signatures). Every function returns rrs_status_t and requires a valid session except registration and login.

Function	Auth required	Success	Errors
auth_register(email, pwd, name)	None	OK	ERR_INVALID_INPUT, ERR_DUPLICATE
auth_login(email, pwd) → session	None	OK + session	ERR_AUTH_FAILED

train_create / update / retire(session, train)	Admin	OK	ERR_FORBIDDEN, ERR_SESSION_EXPIRED, ERR_CAPACITY_CONFLICT
search_routes(src, dst, date, opts) → routes	Passenger	OK + routes	ERR_INVALID_INPUT, ERR_NO_ROUTE
fare_calculate(route, class, concession) → fare	Passenger	OK	ERR_INVALID_INPUT
eco_score(route) → score	Passenger	OK	ERR_INVALID_INPUT
booking_create(session, route, class, idempotency_key) → booking	Passenger	OK + PNR/waitlist	ERR_PAYMENT_FAILED, ERR_SESSION_EXPIRED
booking_cancel(session, pnr)	Passenger (owner)	OK	ERR_NOT_FOUND, ERR_FORBIDDEN
waitlist_promote(train_id, class)	System (internal)	OK	—

API contract rule: signatures are frozen after Sprint 1 review; any change requires agreement from all four module owners and an updated Appendix C traceability entry.

Idempotency: *booking_create() accepts an idempotency key for retry-safe booking requests. If a request with an already processed key is received, the existing booking result is returned instead of creating a duplicate booking.*

4.4 Error Handling, Logging & Monitoring

Every function returns a status code from a single enum so no function crashes the TUI on bad input. The TUI maps each code to a user-facing message; any non-OK code returned from a protected (session-checked) call is written to the audit log with timestamp, user id and function name. No passwords, tokens or mock-payment details are ever logged.

Code	Meaning
ERR_INVALID_INPUT	Failed field validation
ERR_AUTH_FAILED	Wrong credentials
ERR_FORBIDDEN	Role not permitted for this action
ERR_SESSION_EXPIRED	Token invalid or expired
ERR_NOT_FOUND	Unknown record (e.g. PNR)
ERR_DUPLICATE	Resource already exists (e.g. email)
ERR_CAPACITY_CONFLICT	Capacity below confirmed bookings (FR-011)
ERR_PAYMENT_FAILED	Mock payment declined
ERR_NO_ROUTE	No path found between stations
ERR_IO	File/DB read or write failure

4.5 UX Design (TUI)

- Persistent header showing the logged-in user and role on every screen.
- Numbered/arrow-key menu navigation with a consistent Back/Cancel key across all screens.
- Inline validation messages shown immediately below the offending field.
- A new passenger can complete search-to-booking within 6 primary screens (NFR-004).
- Waitlist position and booking status are always visible on the tracking screen.

4.6 Data / Class Model

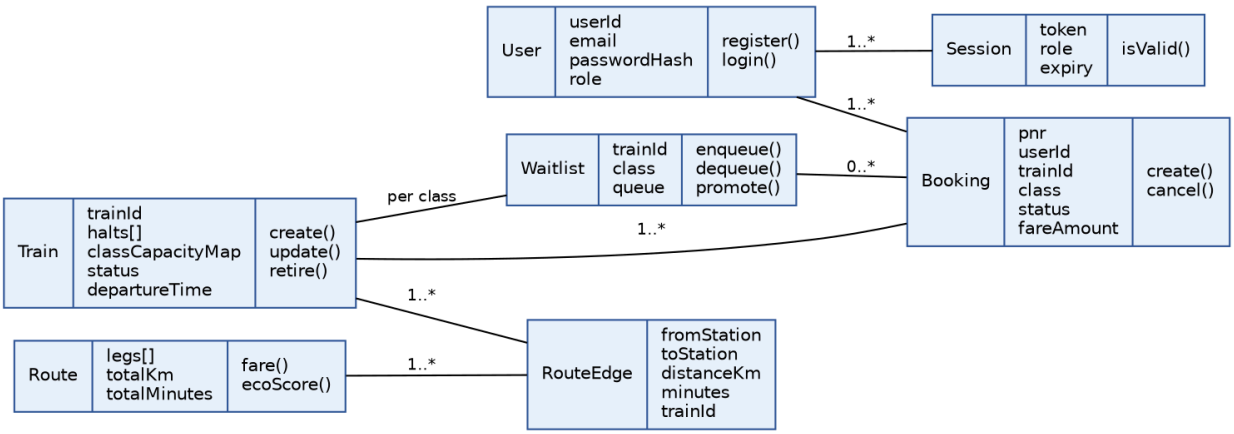


Figure 5: Domain class model — User, Session, Train, Route/Edge, Booking, Waitlist

4.7 Activity Diagram — Book Ticket

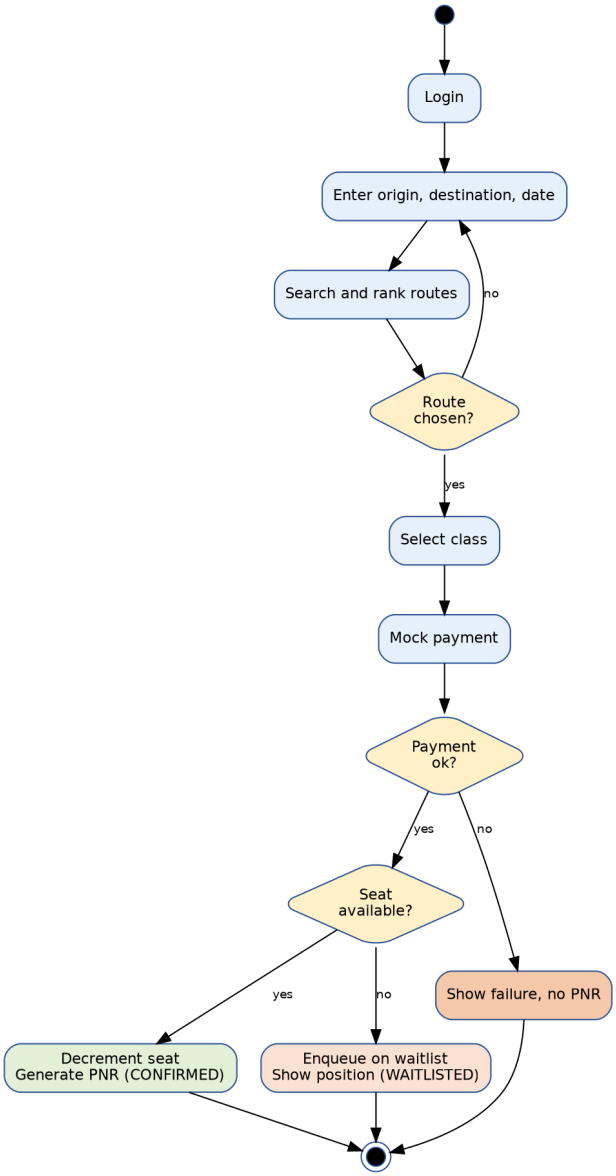


Figure 6: Activity diagram for the Book Ticket workflow

4.8 Deployment View

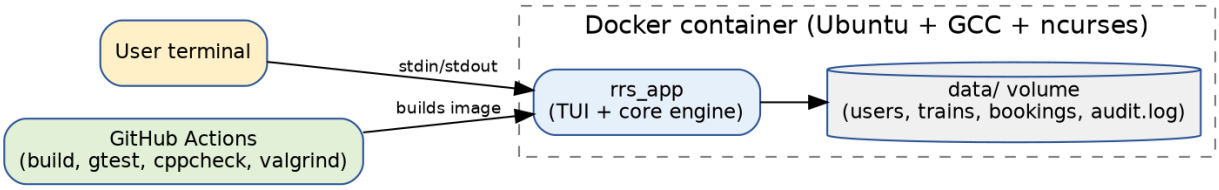


Figure 7: Deployment — Docker container running the TUI and core engine, built and tested via GitHub Actions

4.9 Open Issues & Next Steps

- Finalize whether the optional MERN/API wrapper is built, pending faculty approval of cross-stack integration.
- Use an adjacency-list representation for the route graph. B-tree structures are used for tree-based train/schedule lookup as required by the SRS.
- Future enhancement: live seat-map visualization and multi-currency fares (out of current scope).

5. Appendices

5.1 Glossary

PNR, RBAC, TUI, Waitlist, Multi-leg journey, Eco-score, Halt, Class — see SRS Appendix A.

5.2 References

- PES University, Jackfruit – Software Engineering Project Guidelines 2026 (UE24CS341A)
- PES University, Software Engineering – Level 3 Mini-Project Student Submission Guide & Exemplar
- Team 5, Railway Reservation System – SRS v1.1
- ncurses Programming HOWTO

5.3 Tools

Graphviz / draw.io / Mermaid (UML diagrams), GitHub Actions (CI/CD), Docker (containerization), cppcheck (static analysis).

SAD v1.1 Revision: Design Changes During Implementation (13 November 2026)

Version 1.0 of this document fixed the architecture and module APIs before implementation (section 4.3 froze the signatures after the Sprint 1 review). This revision records every change made during Sprints 1 to 3, with the reason. The layered architecture with a facade (section 3.5), the four modules and their owners, the error codes (section 4.4) and the security architecture (section 3.9) are unchanged.

Revision History

Date	Change	Version
01.10.2026	Initial architecture and design specification	1.0
13.11.2026	Design changes made during implementation recorded (this page); traceability updated	1.1

1. API changes (section 4.3)

Function	Designed (v1.0)	Implemented	Reason	Req
`booking_create`	(session, route, class, idempotency_key)	(session, train_id, class, fare_amount, idempotency_key, out)	A connecting journey is several tickets on different trains, each with its own seats and waitlist. `facade_book` books every leg and rolls back earlier legs if a later one fails. The facade always passes `fare_calculate()`'s value, never user input.	FR-018, FR-024, SEC-003
`eco_score`	(route) → score	(route, class, out) → {co2_kg, car_co2_kg, score}	CO2 per passenger depends on the travel class; the TUI shows kg of CO2 next to the score.	FR-015
`search_rank`	-	(routes, class, concession, sort_key, filter, out)	Sort and filter an existing result without a new search (SRS 5.3.2).	FR-016
`search_default_opts`	-	max legs 2, max results 20, minimum transfer 30 min	Documented defaults for the opts argument.	FR-012, FR-013
`search_routes`	requires a passenger session	no session argument; the facade checks the session first	Keeps the search engine independent of auth so it is testable on its own.	NFR-006, NFR-007
Auth	auth_register, auth_login	+ auth_authorize, auth_authorize_for, auth_logout, auth_profile_get, auth_profile_update	RBAC on every call, logout, profiles, audited admin calls.	FR-004 to FR-006, SEC-006
Inventory	train_create / update / retire	+ train_get, train_is_bookable	Booking reads a train's capacity and status.	FR-009, FR-018
Facade	single entry point	facade_init, facade_seed_demo, facade_search, facade_book, facade_my_bookings, facade_cancel	As designed for the passenger flow. The two admin screens call train_create / train_retire directly; those functions check session and role themselves.	SEC-002

2. Other design changes

Area (SAD section)	Designed	Implemented	Reason / effect
Persistence (3.4, 3.6)	Flat files or an embedded database behind a common interface	In memory; only the audit log is written to a file	Time went into the TUI. Each module keeps its data behind its own functions, so a store can be added in persistence/store.c without API changes.
Train lookup (4.9)	B-tree for train and schedule lookup	Sorted array with binary search (inventory.c); hash map station → stops for the route graph	Same O(log n) lookup for up to 1,024 trains with much less code; adjacency-list route graph as designed.

Area (SAD section)	Designed	Implemented	Reason / effect
Audit logging (4.4)	Every non-OK result of a protected call is logged	Failed logins, rejected or expired sessions, denied and successful admin actions are logged	Not yet logged: a passenger denied access to another passenger's booking; ADMIN_ACTION is written when the role check passes, before the action runs.
TUI (4.5)	Header shows user and role; validation messages under each field; screens for all features	Header shows app and screen name; messages in the status line; no profile, train-edit or filter screens	Search to booking takes 5 screens (NFR-004 allows 6). Missing screens' features work and are tested through the API.
Demo admin	-	A demo admin account (admin@rrs.in) is created at start-up	Lets the admin screens be shown without a database; must be removed for real use.
Concurrency (4.1)	Race-free waitlist promotion	One mutex around every booking and waitlist operation, including the session check	Verified with ThreadSanitizer and a 50-thread test (NFR-009).
Deployment (4.8, Fig. 7)	Docker container built by GitHub Actions	GitHub Actions builds and tests on Ubuntu; no Docker image	Every team member builds natively on Linux/WSL with the same package list as CI.
Open issue (4.9)	Optional MERN/API wrapper	Not built	The ncurses TUI covers every use case.
Schedule	Four sprints	Two sprints plus an optional Sprint 3 for spill-over (TUI, thread safety, test reports)	As permitted by the course's Part-3 deliverables.

Full detail: `docs/design_deviations.md`. Requirement traceability: SRS v1.2 Appendix C.